

Aptitude Preparation Platform

Complete Project Specification Document

Project Name	Personalized Aptitude Preparation Platform for Placements
Document Type	Full Technical and Functional Specification
Target Users	Students preparing for campus placements + Admins/Teachers
Purpose	AI-ready specification — upload to Claude/Cursor/Copilot to generate the project
Project Type	Full-stack web application (Student portal + Admin portal)
Differentiator	AI-powered personalization, adaptive practice, smart proctoring

How to use this document: Upload this PDF to Claude (or any LLM-based coding assistant). Ask it to build the project module-by-module following the phases outlined in Section 11. Each section is self-contained — you can also feed individual sections for focused development.

Table of Contents

1. Project overview and objectives
2. User roles and authentication
3. Student portal — complete module breakdown
4. Admin portal — complete module breakdown
5. AI and personalization engine
6. Proctoring system specification
7. Database schema (all tables and relations)
8. API endpoint specification (REST)
9. Technology stack and architecture
10. UI/UX design guidelines
11. Development phases and milestones
12. Security, performance, and deployment
13. Future enhancements
14. Build instructions for AI assistants

1. Project Overview and Objectives

This platform is a complete placement aptitude preparation system with two portals: a Student portal for learning and practice, and an Admin/Teacher portal for content management and student tracking. The platform's core differentiator is AI-powered personalization — it analyzes each student's weak areas and creates a custom study plan, generates adaptive question sets, and offers a fully proctored test environment.

1.1 Primary objectives

- Provide a single platform for aptitude prep across Quantitative, Logical Reasoning, Verbal, and Data Interpretation
- Identify each student's weak topics automatically through practice and tests
- Generate personalized weekly study plans based on weak areas
- Allow admins to create tests manually or through AI from a syllabus or topic
- Enforce strict proctoring during formal tests (fullscreen lock, tab-switch detection, copy-paste disabled)
- Provide company-wise preparation resources (past papers, important topics, patterns)
- Track student progress through detailed analytics and reports

1.2 Target users

- Students — final-year college students preparing for campus placements
- Teachers — faculty members who create tests, upload materials, and monitor students
- Admins — system administrators who manage users, batches, and platform settings

1.3 Key features at a glance

Module	Student	Admin
Study materials (PDFs, videos)	View / download	Upload / organize
Teacher notes viewer	In-app PDF viewer	Upload notes
Self-practice	Topic-based or syllabus upload	—
Admin tests	Take in practice or proctored mode	Create manually or via AI
Reports	Personal analytics	Per-student and aggregate
Personalized plan	Auto-generated, weak area focused	Review and customize
Past year question papers	Company-wise download	Upload and tag
Dashboard	Personal progress view	Platform-wide analytics
Proctoring	Subject to during tests	Configure settings

2. User Roles and Authentication

2.1 Roles

- **Student** — accesses learning materials, takes practice and tests, views personal reports
- **Teacher** — creates and manages tests, uploads materials, views student reports for assigned batches
- **Admin** — full system access, manages teachers/students/batches, platform settings, audit logs

2.2 Authentication flow

- **Registration** — name, email, password, role (student by default; teacher/admin invite-only), college, branch, year, target companies
- **Email OTP verification** — 6-digit code valid for 10 minutes, sent via SMTP (Nodemailer/Django-mail)
- **Login** — email + password; JWT token issued (access + refresh)
- **Forgot password** — email-based reset link, token valid 1 hour
- **Change password** — requires current password
- **Logout** — invalidates refresh token (store blacklist in Redis)
- **Session management** — access token (15 min), refresh token (7 days), auto-refresh on 401
- **Role-based middleware** — protect routes by role; students cannot access admin endpoints

2.3 User profile fields

Field	Type	Required	Description
user_id	UUID	Yes	Auto-generated
name	String(100)	Yes	Full name
email	String(150)	Yes	Unique, verified
password_hash	String	Yes	bcrypt hashed
role	Enum	Yes	student, teacher, admin
college	String(200)	Student	College name
branch	String(50)	Student	CSE, IT, ECE, etc.
year	Integer	Student	1-4
target_companies	JSON array	Student	TCS, Infosys, Wipro, etc.
target_exam_date	Date	Student	Optional placement date
profile_photo	URL	No	S3/Cloudinary URL
batch_id	UUID FK	Student	Linked to batch
created_at	Timestamp	Yes	Auto
last_login	Timestamp	No	Updated on login
is_verified	Boolean	Yes	Email verified
is_active	Boolean	Yes	Soft delete flag

3. Student Portal — Complete Module Breakdown

3.1 Dashboard (Home)

The first screen after student login. Should give an instant snapshot of progress and next actions.

Components:

- **Top stat cards** — current streak, today's plan progress %, weak area alert, next scheduled test
- **Quick action buttons** — Continue practice, Daily challenge (5 questions), View plan
- **Accuracy trend chart** — last 30 days line chart (Chart.js or Recharts)
- **Topic-wise heatmap** — color-coded grid showing strength per topic
- **Recent activity feed** — last 5 tests/practices with scores
- **Notifications panel** — new test assigned, plan updated, announcement
- **Leaderboard preview** — current rank in class/college

3.2 Study Materials

Centralized learning library, organized hierarchically.

Structure:

- **Level 1 — Subject** — Quantitative Aptitude, Logical Reasoning, Verbal Ability, Data Interpretation
- **Level 2 — Topic** — e.g., under Quant: Percentages, Time and Work, Probability, Algebra
- **Level 3 — Concept** — e.g., under Time and Work: Pipes and Cisterns, Combined Work, Efficiency Problems

Per concept, the student sees:

- Concept explanation video (YouTube embed)
- Shortcut tricks PDF (in-app PDF.js viewer with zoom, search, page bookmarks)
- Formula sheet PDF
- Solved examples PDF
- 'Mark as learned' button — updates progress and feeds the personalization engine
- 'Practice this concept' button — jumps to focused practice session

Features:

- Global search across all materials (title, content)
- Filter by subject, topic, difficulty, company-relevance
- Bookmark any material
- Recent items section
- Download permission controlled by admin per file

3.3 Teacher Notes Viewer

Separated from study materials because teachers upload these specifically and they're often batch-specific.

- **In-app PDF viewer** — using PDF.js, no need to leave the platform
- **Zoom, pan, search, page navigation**
- **Highlight text** — saved per-student
- **Add personal notes** — sticky notes attached to specific pages
- **Download toggle** — admin chooses per file whether download is allowed
- **Filter by teacher, batch, subject**
- **Recently uploaded** section at top

3.4 Practice Session

Three ways to start a practice session:

Method 1: Topic-based custom test

Flow:

- Step 1 — Student selects subject (Quant/Logical/Verbal/DI)
- Step 2 — Topics in that subject are displayed as checkboxes
- Step 3 — On topic selection, all concepts under that topic appear (multi-select)
- Step 4 — Student chooses: number of questions (10/20/30/50/custom), difficulty (easy/medium/hard/mixed), time limit (yes/no)
- Step 5 — System fetches matching questions from question bank and starts the session

Method 2: Syllabus upload (AI-powered)

Flow:

- Step 1 — Student uploads syllabus (PDF or DOCX)
- Step 2 — AI extracts topics using OCR (Tesseract) + NLP (keyword matching against topic taxonomy, or LLM extraction via Claude/OpenAI API)
- Step 3 — Extracted topics displayed for confirmation (student can add/remove)
- Step 4 — Student selects topics to practice
- Step 5 — AI generates a question set covering selected topics from the question bank (or generates new ones via LLM)
- Step 6 — Practice session starts

Method 3: Quick practice (recommended addition)

- 'Continue where I left off' — resumes incomplete session
- 'Practice my weakest topic' — auto-picks based on personalization engine output
- 'Random mixed practice' — 20 random questions across all topics

During the practice session:

- Question display with 4 options (or numeric input for some)
- Per-question timer (optional, configurable)

- Instant feedback after submission (correct/incorrect + detailed solution)
- 'Explain like I'm 5' button — LLM-powered simplified explanation
- Bookmark for later review
- 'Skip and come back' option
- Progress bar showing X/N questions answered
- Pause/resume capability (practice mode only)
- End-of-session summary with score, time, topic-wise breakdown

3.5 Admin Tests (Assigned Tests)

Tests created by teachers/admins, assigned to the student's batch.

- **Two tabs** — Upcoming and Completed
- **Card view** — title, mode (practice/test), duration, total questions, deadline, status
- **Two modes:**
 - **Practice mode** — fullscreen only, instant feedback, can pause, retake allowed
 - **Test mode** — full proctoring (see Section 6), no pause, results shown only after submission and per admin policy
- **Pre-test instructions screen** — duration, marking scheme, proctoring rules, 'I agree' checkbox
- **Auto-save** every 10 seconds to prevent data loss on disconnect
- **Auto-submit** when timer ends

3.6 Student Reports

Two tabs: Practice Reports and Test Reports.

Per session, the report shows:

- Overall score, accuracy %, time taken, percentile (vs other students)
- Topic-wise breakdown — accuracy and time per topic
- Question-by-question view — your answer, correct answer, time spent, solution explanation
- Strength and weakness analysis with bar charts
- 'Similar questions' button — drill the concepts you got wrong
- Comparison vs class average and your previous attempts
- Downloadable PDF report

3.7 Personalized Study Plan

This is the platform's flagship feature — the AI-driven plan generator.

Flow:

- Trigger: after any practice or test, weak areas are visible in the report
- Student clicks 'Generate Plan' button on weak area
- AI analyzes accuracy patterns across topics and creates a week-wise plan

Each week of the plan contains:

- Topics to cover (with deep links to study materials and videos)
- Practice tests for those topics — auto-generated, takeable inline
- Daily targets (e.g., 'Mon: 30 min video + 20 questions on Probability')
- Progress bar (% complete)
- End-of-week re-evaluation test — confirms whether the weak area is now fixed
- If still weak — plan auto-extends with more advanced material

Plan adjustments:

- 'Skip today' — shifts pending tasks forward
- 'Add more practice' — student requests more questions on a topic
- 'Reset plan' — generates a fresh plan from latest weak areas
- Admin can review and edit any AI plan (see admin section)

3.8 Past Year Question Papers (Company Corner)

Company-wise placement preparation resources.

Flow:

- Student selects company (TCS, Infosys, Wipro, Accenture, Capgemini, Cognizant, etc.)
- Page shows: important topics for that company with weightage %, test pattern (sections, duration, marking), last 3-5 years' question papers (PDF download), company-specific mock test (built from past patterns), cutoff trends, interview tips
- Filter by year, exam round (Round 1, Round 2)
- Bookmark favorite companies

3.9 Recommended Extras

- **Doubt forum** — student posts questions, peers and teachers answer, teachers can mark 'best answer' (massive engagement driver)
- **Mistake notebook** — every wrong answer auto-saved, student revisits before exams
- **Daily challenge** — 5 questions a day, builds streak, gamification
- **Leaderboard** — class, college, all-India rank, filterable by subject
- **Achievements / badges** — '100-day streak', 'Probability master', 'Top 10%', incentivizes consistent use
- **Bookmarks** — save any question for later

4. Admin Portal — Complete Module Breakdown

4.1 Admin Dashboard

- **Top stats** — total students, active today (DAU), tests scheduled this week, average score trend
- **Charts** — student engagement (DAU/MAU graph), test completion rates, topic-wise difficulty (where students fail most), top 10 and bottom 10 performers this week
- **Alerts panel** — students inactive 7+ days, students with declining performance, students who completed all assignments
- **Quick actions** — create new test, upload material, send announcement, view recent reports

4.2 Test Builder

The most critical admin feature. Multi-step wizard.

Step 1: Test setup

Field	Description
Title	Test name (e.g., 'TCS Mock Test 1')
Description	Optional context
Mode	Practice or Test
Assign to	Whole batch, specific students, or 'all'
Start time	When test becomes available
End time	Deadline
Duration	Total time in minutes
Total marks	Sum of all questions
Marking scheme	+1/-0.25, or custom per question
Shuffle questions	Yes/No
Shuffle options	Yes/No
Show results	Immediately / After deadline / Manual

Step 2: Question source (three methods, can be mixed)

- **Manual entry** — type question, 4 options, mark correct answer, add solution/explanation, tag topic + concept + difficulty
- **Pick from question bank** — search and filter existing questions, add to test in bulk
- **AI-generated** — three sub-options:
 - **Single topic** — choose topic, number of questions, difficulty distribution, generate
 - **Multi-topic with weightage** — e.g., 40% Quant, 30% Logical, 30% Verbal, AI distributes accordingly
 - **Upload syllabus** — AI extracts topics from PDF, admin confirms, AI generates questions

Step 3: Review and edit

Critical step — admin sees all generated questions in a list. Can edit any question text, change options, mark different correct answer, delete, or click 'Regenerate' for AI-made ones. Without this

step, AI-generated quality drops sharply.

Step 4: Proctoring settings (test mode only)

- Force fullscreen — exit fullscreen counts as violation
- Tab switch detection — log every switch, auto-submit after N violations (admin sets N, typically 3)
- Disable copy/paste, right-click, dev tools
- Webcam capture — snapshot every 30 seconds (optional, requires student consent)
- Face detection — flag if no face detected for 30+ seconds
- Multiple-face alert — flag if 2+ faces detected (suggests external help)
- Screen recording flag (optional)
- Network disconnection handling — auto-save every 10s, resume on reconnect within 5 min

Step 5: Publish

Schedule for future or publish immediately. Assigned students get notification + email.

4.3 Question Bank

- Master pool of all questions across the platform
- Each question tagged with: subject, topic, concept, difficulty (1-5), company tags (TCS, Infosys, etc.), question type (MCQ, numeric, true/false)
- Bulk import via Excel/CSV (admin-friendly)
- Duplicate detection — warn if similar question exists
- Quality flags — questions marked 'unclear' by students get flagged for review
- Version control — keep edit history
- Bulk operations — assign tags, delete, export

4.4 Material Library

- Centralized upload for all PDFs, videos (YouTube URLs), notes, past papers
- Folder structure mirrors student view
- Tag every upload — subject, topic, concept, difficulty, company-relevance, target batch
- Visibility — public / batch-specific / individual student
- Download permission toggle per file
- Version control for updated materials
- Bulk upload via ZIP

4.5 Reports Section (Admin)

Filterable views by batch, student, test, topic, date range.

Per-test view:

- Full leaderboard with rank, score, time, percentile
- Question-wise difficulty analysis — % who got each question right

- Time analysis — average time per question
- Topic-wise performance across all attendees
- Export to Excel/PDF

Per-student view:

- Complete attempt history
- Strength and weakness profile
- Plan adherence — % of plan tasks completed
- Proctoring violations history (if any)
- Engagement metrics — login frequency, time spent

4.6 Personalized Plan Management

- Per-student view of their current AI-generated plan
- Edit any week — add topic, add custom material link, add custom test, remove items
- Lock weeks — mark mandatory, student can't skip
- Bulk plan assignment — assign one plan to entire batch
- Plan templates — save commonly used plan structures
- Override AI — for special cases (slow learners, advanced students)

4.7 User Management

- List all students/teachers with search, filter, sort
- Create batches (e.g., 'CSE 2025 Batch A')
- Assign students to batches
- Bulk import students via Excel
- Reset password for any user
- Suspend / reactivate accounts
- Role assignment (promote teacher → admin)

4.8 Doubt Management and Announcements

- **Doubt management** — see all student doubts, filter by status (open/answered/resolved), assign to teachers, post answers, mark 'best answer'
- **Announcements** — push notifications to selected batches, schedule for future, attach files
- **Audit log** — every admin action logged (who created/edited/deleted what, when) for accountability

5. AI and Personalization Engine

5.1 Three approaches (start simple, evolve)

Approach 1: Rule-based (Start here, ship in 2 weeks)

- Tag every question with subject, topic, concept, difficulty (1-5)
- Track per-user accuracy per topic in a `student\_skill` table
- Update accuracy after every answer
- Rules:
 - If accuracy on topic X < 60% — mark as weak, queue easy questions next
 - If accuracy 60-80% — queue medium questions
 - If accuracy > 80% — queue hard questions
- Simple to implement, works well enough for v1

Approach 2: Elo / IRT-based (v2)

- Each user has an Elo rating per topic (start at 1200)
- Each question has a difficulty Elo rating
- After each answer, update both ratings using Elo formula
- Question selector picks questions near the user's rating ± 100
- This is how Chess.com and Duolingo work
- Provides smooth, adaptive difficulty

Approach 3: ML-based (v3, after collecting data)

- Train collaborative filtering or neural net on attempt logs
- Predict probability that a user gets a given question correct
- Recommend questions in the 'just-right' zone ($P(\text{correct}) \approx 0.65-0.75$)
- Use TensorFlow/PyTorch with scikit-learn baselines
- Needs ~1000+ users generating data first

5.2 Weak area detection algorithm (pseudocode)

```
FOR each topic IN student_topics:
    recent_attempts = last 20 attempts in topic
    accuracy = correct_count / total
    avg_time = mean(time_taken)
    IF accuracy < 60% OR avg_time > expected_time × 1.5:
        mark topic AS weak
        weight = (60 - accuracy) + (time_overshoot factor)
SORT topics by weight DESCENDING
RETURN top 5 weak topics
```

5.3 Plan generation algorithm

- Input: list of weak topics with severity scores, target exam date, available study hours/day
- Step 1 — Distribute topics across weeks (most severe first)
- Step 2 — For each topic, allocate days: 1 day video + reading, 2 days practice, 1 day re-test

- Step 3 — Generate daily tasks with realistic time estimates
- Step 4 — Insert re-evaluation tests at end of each week
- Step 5 — Store plan in database, send to frontend

5.4 AI question generation (LLM-based)

For 'AI-generated' option in test builder and syllabus-upload practice:

Prompt template (send to Claude/OpenAI API):

```
You are an aptitude question generator. Generate {N} multiple choice questions on the
topic '{topic}' under '{subject}'. Difficulty: {difficulty}. Format each question as
JSON: {
  'question': '...',
  'options': ['A', 'B', 'C', 'D'],
  'correct_index': 0,
  'explanation': '...',
  'difficulty': 'medium',
  'estimated_time_seconds': 60
}
Return as JSON array. Do not include markdown.
```

Always have admin review AI-generated questions before publishing.

5.5 Syllabus topic extraction

- Step 1 — Extract text from uploaded PDF/DOCX (pdfplumber, python-docx)
- Step 2 — Run text through LLM with prompt: 'Extract aptitude topics from this syllabus. Match against this taxonomy: [list]. Return JSON array of matched topics.'
- Step 3 — Display extracted topics to user for confirmation
- Step 4 — Pull/generate questions for confirmed topics

6. Proctoring System Specification

6.1 Modes

- **Practice mode** — fullscreen required, no other restrictions
- **Test mode** — full proctoring stack active

6.2 Fullscreen enforcement

- On test start — call `document.documentElement.requestFullscreen()`
- Listen for `fullscreenchange` event
- If user exits fullscreen — display warning modal, increment violation count
- After N violations (admin-configured, default 3) — auto-submit test
- Show countdown — 'Return to fullscreen in 10 seconds or test will auto-submit'

6.3 Tab switch detection

- Listen for `visibilitychange` event
- If `document.hidden === true` — log violation with timestamp
- Show warning on return
- After N violations — auto-submit
- Log all violations to database for admin review

6.4 Copy-paste and right-click

```
document.addEventListener('contextmenu', e => e.preventDefault())
document.addEventListener('copy', e => e.preventDefault())
document.addEventListener('paste', e => e.preventDefault())
document.addEventListener('keydown', e => {
  if (e.ctrlKey && ['c','v','x','a','u','s','p'].includes(e.key)) e.preventDefault();
  if (e.key === 'F12') e.preventDefault();
})
```

6.5 Webcam proctoring (optional, advanced)

- Request webcam permission at test start
- Capture snapshot every 30 seconds via `getUserMedia` + canvas
- Upload to S3/storage with `test_id` and timestamp
- Run face-detection (`face-api.js` or backend ML) — flag if no face for 30+ seconds, flag if multiple faces detected
- Admin reviews flagged snapshots after test

6.6 Violation log structure

Field	Type	Description
<code>violation_id</code>	UUID	Unique ID
<code>test_attempt_id</code>	UUID FK	Linked attempt
<code>student_id</code>	UUID FK	Student

violation_type	Enum	fullscreen_exit, tab_switch, copy_attempt, no_face, multi_face
timestamp	DateTime	When it occurred
details	JSON	Extra info (e.g., duration of tab-switch)
snapshot_url	String	Webcam image URL (if applicable)

7. Database Schema (Complete)

Recommendation: PostgreSQL for relational data, MongoDB or PostgreSQL JSON column for attempt logs (high write volume), Redis for sessions and leaderboards.

7.1 users

```
CREATE TABLE users (  
  user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) CHECK (role IN ('student', 'teacher', 'admin')),  
  college VARCHAR(200),  
  branch VARCHAR(50),  
  year INTEGER,  
  target_companies JSONB,  
  target_exam_date DATE,  
  profile_photo_url TEXT,  
  batch_id UUID REFERENCES batches(batch_id),  
  is_verified BOOLEAN DEFAULT FALSE,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT NOW(),  
  last_login TIMESTAMP  
);
```

7.2 subjects, topics, concepts (taxonomy)

```
CREATE TABLE subjects (  
  subject_id UUID PRIMARY KEY,  
  name VARCHAR(100), -- Quantitative, Logical, Verbal, DI  
  display_order INTEGER  
);  
  
CREATE TABLE topics (  
  topic_id UUID PRIMARY KEY,  
  subject_id UUID REFERENCES subjects(subject_id),  
  name VARCHAR(150), -- e.g., 'Time and Work'  
  description TEXT  
);  
  
CREATE TABLE concepts (  
  concept_id UUID PRIMARY KEY,  
  topic_id UUID REFERENCES topics(topic_id),  
  name VARCHAR(150), -- e.g., 'Pipes and Cisterns'  
  description TEXT  
);
```

7.3 questions

```
CREATE TABLE questions (  
  question_id UUID PRIMARY KEY,  
  concept_id UUID REFERENCES concepts(concept_id),  
  question_text TEXT NOT NULL,  
  question_type VARCHAR(20), -- mcq, numeric, true_false  
  options JSONB, -- [{'id':'A','text':'...'}, ...]  
  correct_answer VARCHAR(10), -- 'A' or numeric value  
  explanation TEXT,
```

```

difficulty INTEGER CHECK (difficulty BETWEEN 1 AND 5),
estimated_time_seconds INTEGER,
company_tags JSONB, -- ['TCS', 'Infosys']
source VARCHAR(50), -- manual, ai_generated, past_paper
created_by UUID REFERENCES users(user_id),
quality_flag VARCHAR(20) DEFAULT 'ok',
created_at TIMESTAMP DEFAULT NOW()
);

```

7.4 tests, test_questions

```

CREATE TABLE tests (
  test_id UUID PRIMARY KEY,
  title VARCHAR(200),
  description TEXT,
  mode VARCHAR(20), -- practice / test
  duration_minutes INTEGER,
  total_marks DECIMAL,
  marking_scheme JSONB, -- {correct:1, wrong:-0.25}
  shuffle_questions BOOLEAN,
  shuffle_options BOOLEAN,
  proctoring_config JSONB, -- {fullscreen:true, tab_switch_limit:3, webcam:false}
  start_time TIMESTAMP,
  end_time TIMESTAMP,
  assigned_to JSONB, -- {batch_ids:[], student_ids:[]}
  created_by UUID REFERENCES users(user_id),
  status VARCHAR(20) DEFAULT 'draft' -- draft, scheduled, live, completed
);

CREATE TABLE test_questions (
  test_id UUID REFERENCES tests(test_id),
  question_id UUID REFERENCES questions(question_id),
  display_order INTEGER,
  marks DECIMAL,
  PRIMARY KEY (test_id, question_id)
);

```

7.5 test_attempts, attempt_answers

```

CREATE TABLE test_attempts (
  attempt_id UUID PRIMARY KEY,
  test_id UUID REFERENCES tests(test_id),
  student_id UUID REFERENCES users(user_id),
  started_at TIMESTAMP,
  submitted_at TIMESTAMP,
  time_taken_seconds INTEGER,
  score DECIMAL,
  total_marks DECIMAL,
  accuracy_percent DECIMAL,
  status VARCHAR(20), -- in_progress, submitted, auto_submitted
  violations_count INTEGER DEFAULT 0
);

CREATE TABLE attempt_answers (
  attempt_id UUID REFERENCES test_attempts(attempt_id),
  question_id UUID REFERENCES questions(question_id),
  selected_answer VARCHAR(10),
  is_correct BOOLEAN,

```

```

    time_taken_seconds INTEGER,
    marked_for_review BOOLEAN,
    PRIMARY KEY (attempt_id, question_id)
);

```

7.6 student_skill_profile (personalization core)

```

CREATE TABLE student_skill_profile (
    student_id UUID REFERENCES users(user_id),
    topic_id UUID REFERENCES topics(topic_id),
    accuracy_percent DECIMAL,
    total_attempts INTEGER,
    correct_count INTEGER,
    avg_time_seconds INTEGER,
    elo_rating INTEGER DEFAULT 1200,
    is_weak BOOLEAN,
    last_attempted TIMESTAMP,
    PRIMARY KEY (student_id, topic_id)
);

```

7.7 study_plans, plan_tasks

```

CREATE TABLE study_plans (
    plan_id UUID PRIMARY KEY,
    student_id UUID REFERENCES users(user_id),
    generated_at TIMESTAMP,
    duration_weeks INTEGER,
    status VARCHAR(20),
    source VARCHAR(20) -- ai_generated, admin_custom
);

CREATE TABLE plan_tasks (
    task_id UUID PRIMARY KEY,
    plan_id UUID REFERENCES study_plans(plan_id),
    week_number INTEGER,
    day_number INTEGER,
    task_type VARCHAR(30), -- video, pdf, practice, test
    reference_id UUID, -- material_id or test_id
    estimated_minutes INTEGER,
    is_completed BOOLEAN DEFAULT FALSE,
    completed_at TIMESTAMP
);

```

7.8 Additional tables

- **materials** — material_id, title, type (pdf/video/note), file_url, subject_id, topic_id, concept_id, uploaded_by, visibility, download_allowed
- **batches** — batch_id, name, college, year, created_by
- **companies** — company_id, name, logo_url, important_topics (JSON), test_pattern (JSON)
- **past_papers** — paper_id, company_id, year, file_url
- **announcements** — announcement_id, title, content, target_batches, created_by, created_at
- **doubts** — doubt_id, student_id, question_text, status, created_at
- **doubt_answers** — answer_id, doubt_id, answered_by, answer_text, is_best_answer
- **bookmarks** — bookmark_id, student_id, item_type (question/material), item_id

- **violations** — see Section 6
- **audit_log** — log_id, user_id, action, entity_type, entity_id, timestamp, details

8. API Endpoint Specification (REST)

Base URL: /api/v1. All authenticated endpoints require Authorization: Bearer <jwt> header.

8.1 Authentication endpoints

Method	Endpoint	Description
POST	/auth/register	Create new user, send OTP
POST	/auth/verify-otp	Verify email with OTP
POST	/auth/login	Login, returns access + refresh token
POST	/auth/refresh	Get new access token from refresh
POST	/auth/logout	Invalidate refresh token
POST	/auth/forgot-password	Send reset link to email
POST	/auth/reset-password	Reset password using token
POST	/auth/change-password	Change current password
GET	/auth/me	Get current user profile

8.2 Student endpoints

Method	Endpoint	Description
GET	/student/dashboard	Dashboard summary
GET	/student/materials	List materials with filters
GET	/student/materials/:id	Get material details
POST	/student/materials/:id/mark-learned	Mark concept learned
GET	/student/notes	List teacher notes
POST	/student/practice/start	Start practice session
POST	/student/practice/upload-syllabus	Upload syllabus, get topics
POST	/student/practice/submit-answer	Submit answer, get feedback
POST	/student/practice/end	End session, get summary
GET	/student/tests	List assigned tests
POST	/student/tests/:id/start	Start a test attempt
POST	/student/tests/attempts/:id/answer	Save answer
POST	/student/tests/attempts/:id/submit	Submit test
POST	/student/tests/attempts/:id/violation	Report proctoring violation
GET	/student/reports	List all reports
GET	/student/reports/:id	Get detailed report
POST	/student/plan/generate	Generate new plan
GET	/student/plan	Get current plan
POST	/student/plan/tasks/:id/complete	Mark task done
GET	/student/companies	List companies
GET	/student/companies/:id	Get company corner data

GET	/student/leaderboard	Get leaderboard
POST	/student/bookmarks	Bookmark item
POST	/student/doubts	Post a doubt
GET	/student/doubts	List doubts

8.3 Admin/Teacher endpoints

Method	Endpoint	Description
GET	/admin/dashboard	Admin dashboard stats
GET	/admin/users	List all users with filters
POST	/admin/users	Create user
PUT	/admin/users/:id	Update user
DELETE	/admin/users/:id	Soft delete user
POST	/admin/users/bulk-import	Excel bulk import
GET	/admin/batches	List batches
POST	/admin/batches	Create batch
POST	/admin/tests	Create test (manual)
POST	/admin/tests/ai-generate	AI-generate questions
POST	/admin/tests/from-syllabus	Generate from syllabus PDF
GET	/admin/tests	List all tests
PUT	/admin/tests/:id	Edit test
POST	/admin/tests/:id/publish	Publish test
GET	/admin/questions	Question bank list
POST	/admin/questions	Add question
POST	/admin/questions/bulk-import	Excel import
POST	/admin/materials	Upload material
GET	/admin/materials	List materials
DELETE	/admin/materials/:id	Delete material
GET	/admin/reports/tests/:id	Per-test full report
GET	/admin/reports/students/:id	Per-student report
GET	/admin/plans/:student_id	View student plan
PUT	/admin/plans/:plan_id	Edit plan
GET	/admin/violations	Proctoring violations log
POST	/admin/announcements	Send announcement
GET	/admin/doubts	List all doubts
POST	/admin/doubts/:id/answer	Answer doubt
GET	/admin/audit-log	View audit log

9. Technology Stack and Architecture

9.1 Recommended stack

Layer	Choice	Why
Frontend	React + Vite OR Next.js 14	Component-based, large ecosystem
UI library	Tailwind CSS + shadcn/ui	Clean, customizable, modern
Charts	Recharts OR Chart.js	Easy integration, good defaults
State	Zustand or Redux Toolkit	Predictable state management
Backend	Node.js + Express OR Python + FastAPI	FastAPI better for ML integration
Database	PostgreSQL (primary)	Strong relational + JSONB for flexible data
Document DB	MongoDB (optional)	For attempt logs at scale
Cache	Redis	Sessions, leaderboards, rate limiting
File storage	AWS S3 / Cloudinary	PDFs, videos, profile photos
Authentication	JWT + bcrypt	Standard, stateless
Email	SendGrid / Resend / SMTP	OTP, notifications, password reset
AI/LLM	Claude API or OpenAI API	Question generation, syllabus extraction
PDF processing	PDF.js (frontend), pdfplumber (backend)	Syllabus parsing, in-app viewing
Proctoring	browser APIs + face-api.js	Built-in browser features + ML
Deployment	Frontend: Vercel; Backend: Railway/Render	Renderers for MVP
Domain & SSL	Cloudflare / Let's Encrypt	Free SSL, DDoS protection

9.2 Folder structure

```
aptitude-platform/
├── frontend/
│   ├── src/
│   │   ├── components/ (UI components)
│   │   ├── pages/student/
│   │   ├── pages/admin/
│   │   ├── pages/auth/
│   │   ├── services/api.js
│   │   ├── stores/ (Zustand)
│   │   ├── hooks/
│   │   ├── utils/
│   │   ├── App.jsx
│   │   ├── package.json
│   │   └── vite.config.js
│   └── backend/
│       ├── src/
│       │   ├── routes/ (auth, student, admin)
│       │   ├── controllers/
│       │   ├── services/ (business logic)
│       │   ├── models/ (DB models)
│       │   ├── middleware/ (auth, rbac)
│       │   ├── ai/ (LLM integration, plan generator)
│       │   ├── utils/
│       │   └── server.js
│       ├── migrations/
│       └── seeds/
```

```
■ ■■■ package.json
■■■ docs/
■■■ README.md
```


10. UI/UX Design Guidelines

10.1 Design principles

- **Clarity over cleverness** — students should never wonder where to click
- **Mobile-responsive** — many students will use phones
- **Dark mode** — many study at night, reduces eye strain
- **Loading states** — skeleton loaders for every async element
- **Error handling** — friendly error messages with retry buttons
- **Empty states** — when no data, show helpful illustration + next action
- **Consistent navigation** — fixed sidebar on desktop, bottom nav on mobile

10.2 Color palette suggestion

Purpose	Color	Hex
Primary (links, buttons)	Blue	#1d4ed8
Success (correct answers)	Green	#16a34a
Warning (alerts)	Amber	#d97706
Danger (wrong, errors)	Red	#dc2626
Neutral text	Gray-900	#111827
Background light	Gray-50	#f9fafb
Background dark	Gray-900	#0f172a

10.3 Key screens to design first

- Login / Register
- Student dashboard
- Practice session setup
- Test taking interface (most critical — fullscreen, timer, question display)
- Report detail view
- Personalized plan view
- Admin dashboard
- Test builder wizard (multi-step)
- Question bank management

11. Development Phases and Milestones

Build in phases. Don't try to ship everything at once.

Phase 1 (Weeks 1-3) — Foundation

- Setup project structure (frontend + backend)
- Database design and migrations
- Authentication (register, login, OTP, JWT)
- Role-based access middleware
- Basic student dashboard (skeleton)
- Basic admin dashboard (skeleton)
- Deploy to staging environment

Phase 2 (Weeks 4-6) — Question Bank and Practice

- Subject/topic/concept taxonomy seed data
- Question bank CRUD (admin)
- Bulk import questions via Excel
- Topic-based practice session (student)
- Submit answer + instant feedback
- Basic report after practice session

Phase 3 (Weeks 7-9) — Tests and Proctoring

- Test builder (manual question entry)
- Test assignment to batches
- Test taking interface for students
- Fullscreen + tab switch detection
- Copy-paste / right-click prevention
- Auto-save and auto-submit logic
- Violation logging
- Test report (per attempt + per test)

Phase 4 (Weeks 10-12) — Personalization

- Student skill profile tracking
- Weak area detection algorithm (rule-based)
- Plan generation logic
- Plan UI for student (week-wise tasks)
- Admin plan management
- Re-evaluation tests

Phase 5 (Weeks 13-15) — AI Features

- Integrate LLM API (Claude/OpenAI)
- AI question generation in test builder
- Syllabus upload + topic extraction
- AI-generated practice from syllabus
- Admin review screen for AI questions

Phase 6 (Weeks 16-18) — Materials and Company Corner

- Material upload (PDFs, videos, notes)
- Student material viewer with PDF.js
- Bookmarks and mistake notebook
- Company corner (TCS, Infosys, etc.)
- Past papers upload and download

Phase 7 (Weeks 19-20) — Engagement and Polish

- Daily challenge
- Leaderboard
- Achievements / badges
- Doubt forum
- Announcements
- Audit log
- Mobile responsive polish
- Dark mode

Phase 8 (Weeks 21-22) — Production Readiness

- Security audit (OWASP top 10)
- Performance optimization (caching, lazy loading)
- Load testing
- Backup strategy
- Monitoring (Sentry, LogRocket)
- Documentation
- Production deployment

12. Security, Performance, and Deployment

12.1 Security checklist

- Passwords hashed with bcrypt (salt rounds ≥ 10)
- JWT tokens with short expiry (15 min access, 7 day refresh)
- HTTPS everywhere (Let's Encrypt free SSL)
- Input validation on all endpoints (Joi/Zod/Pydantic)
- SQL injection prevention (parameterized queries, ORM)
- XSS prevention (escape user content, CSP headers)
- CSRF protection (SameSite cookies, CSRF tokens for forms)
- Rate limiting (Redis-backed, e.g., 100 req/min per IP)
- File upload restrictions (size, type, virus scan)
- Environment variables for secrets (never commit .env)
- Role-based access control on every endpoint
- Audit logging for all admin actions
- GDPR-style account deletion option

12.2 Performance

- Database indexes on frequently queried columns (user_id, test_id, topic_id)
- Pagination on all list endpoints (limit + cursor)
- Redis caching for leaderboards, dashboard stats (TTL 5 min)
- CDN for static assets and PDFs (Cloudflare)
- Lazy loading for charts and images
- Code splitting in frontend (React.lazy)
- WebSocket or polling for real-time test sync (auto-save)
- Background jobs for heavy tasks (plan generation, report export)

12.3 Deployment

- **Frontend** — Vercel (auto-deploy from GitHub main branch)
- **Backend** — Railway or Render (Docker container, auto-scale)
- **Database** — Neon (Postgres) or Supabase free tier for MVP, AWS RDS for production
- **Storage** — AWS S3 or Cloudflare R2 for files
- **CI/CD** — GitHub Actions: run tests, build, deploy on merge to main
- **Monitoring** — Sentry (errors), LogRocket (sessions), UptimeRobot (availability)
- **Backups** — daily DB snapshots, retained 30 days

13. Future Enhancements (Post-MVP)

- **Mobile apps** — React Native for iOS/Android
- **IRT-based adaptive difficulty** — more sophisticated than rule-based
- **ML recommender** — after 1000+ users generate data
- **Live classes** — Jitsi/Zoom integration for teacher sessions
- **Group study rooms** — students practice together
- **AI tutor chat** — Claude-powered chatbot to explain any concept
- **Voice-based mock interviews** — AI conducts HR + technical interview practice
- **Resume builder** — placement-ready resumes
- **Job board** — companies post openings, students apply
- **Multi-language support** — Tamil, Hindi, Telugu for vernacular students
- **Parent dashboard** — if targeting school market
- **Payment gateway** — Razorpay/Stripe for premium plans
- **White-label option** — colleges can rebrand for their students

14. Build Instructions for AI Assistants

How to feed this document to Claude/Cursor/Copilot to build the project:

14.1 Recommended approach

- **Don't ask AI to build everything in one prompt** — quality drops sharply
- **Build phase by phase** — feed Phase 1 first, get it working, then move to Phase 2
- **Build module by module within each phase**
- **For each module**, give the AI: relevant section from this PDF + the specific files to create + the database tables it needs

14.2 Example prompts to use

After uploading this PDF, use prompts like:

Prompt 1 — Project setup

I've uploaded the full project spec for an aptitude platform. Start with Phase 1 (Foundation). Generate:

1. The folder structure as specified in Section 9.2
2. package.json files for frontend (React + Vite + Tailwind) and backend (Node + Express)
3. PostgreSQL connection setup
4. All migrations for the 'users', 'subjects', 'topics', 'concepts' tables from Section 7
5. Basic Express server with a health check endpoint

Provide a README with setup instructions.

Prompt 2 — Authentication

Now build the complete authentication module per Section 2 and Section 8.1:

1. Register endpoint with email OTP (use nodemailer)
2. Login with JWT (access + refresh)
3. Middleware for role-based access
4. Frontend pages: Register, Login, OTP verify, Forgot password

Use the field structure from Section 2.3.

Prompt 3 — Practice session

Build the practice session module per Section 3.4. I need:

1. Backend endpoints from Section 8.2 (practice/start, submit-answer, end)
2. Topic-based selection UI (multi-select concepts under each topic)
3. Question display with timer
4. Instant feedback after each answer
5. End-of-session summary

Use the 'questions' and 'attempt_answers' tables from Section 7.

Prompt 4 — Test builder with AI

Build the admin test builder per Section 4.2. I need the multi-step wizard:

1. Test setup form with all fields from the table
2. Question source picker (manual / question bank / AI-generated)
3. AI generation using Claude API with the prompt template from Section 5.4
4. Review screen where admin can edit/regenerate questions
5. Proctoring config from Section 6
6. Publish flow

Backend endpoints from Section 8.3.

Prompt 5 — Proctoring

Implement the proctoring system from Section 6 for test mode:

1. Fullscreen enforcement (Section 6.2)
2. Tab switch detection (Section 6.3)
3. Copy-paste prevention (Section 6.4 code snippet)
4. Violation logging to the 'violations' table from Section 6.6
5. Auto-submit after N violations

Build as React hooks I can use in the test page.

14.3 Tips for best results

- Always reference specific sections (e.g., 'per Section 5.2') — the AI will follow them
- After each module, test it end-to-end before moving to the next
- Ask the AI to add comments referencing this spec for traceability
- When AI deviates, paste the relevant section text directly into the chat to refocus it
- Generate seed data after Phase 2 — without questions, you can't test anything
- Use AI to write tests too — 'Generate Jest/Pytest tests for the authentication module'

End of Document

This specification is designed to be self-contained. Combined with an AI coding assistant, it should give you everything needed to build the platform end-to-end. Build in phases, test thoroughly between phases, and iterate based on real student feedback. Good luck!